

IT 2012 - Unstructured Data

Lab 12 - Data Visualization

Assist. Prof. Dr. Dželila Mehanović

Introduction

Data visualization is the process of representing data graphically so that patterns, trends, and insights can be perceived immediately. Effective visualization bridges raw data and human understanding and translates numbers into stories. This lab implements a full-stack visualization module for the Movie Industry Analytics Pipeline, producing both publication-ready static figures and interactive HTML-based exploratory charts.

Learning Objectives

- Understand the matplotlib object-oriented API (Figure / Axes model) and use it to construct multi-panel layouts.
- Apply seaborn to generate statistical visualizations (histograms, boxplots, scatter plots, heatmaps) with minimal boilerplate.
- Use Plotly Express and Plotly Graph Objects to build interactive charts with rich hover tooltips, saved as self-contained HTML files.
- Automate chart generation through a reusable Python module and CLI entry-point script.
- Apply Tufte's principles of graphical excellence: maximize data-ink ratio, avoid chartjunk, choose chart types that match the question being asked.
- Document visualization choices explaining why each chart type was selected for the underlying data characteristic.

This lab introduces a new **src/visualization/** package with dedicated modules for static and interactive chart generation. It also adds a CLI script **scripts/generate_visualizations.py**, a notebook **notebook/lab12_visualization.ipynb**, and two output directories under **outputs/visualizations/**. The pipeline entry point **run_pipeline.py** is updated with a new **run_visualizations_pipeline()** step.

In **src/visualization/** you need to add the following:

- **src/visualization/__init__.py** – makes the visualization folder a Python package and re-exports all chart functions so they can be imported from a single location.
- **src/visualization/static_charts.py** – contains 8 matplotlib and seaborn chart functions. Each function accepts a DataFrame and an output directory, draws a figure using the object-oriented API, and saves it as both PNG (300 dpi) and PDF.
- **src/visualization/interactive_charts.py** – contains 5 Plotly Express and Graph Objects chart functions. Each function saves a self-contained interactive HTML file that can be opened in any browser without a running server.

- **src/visualization/chart_generator.py** – is the orchestrator module. It loads the cleaned dataset, iterates over all static and interactive chart functions, and calls each one in sequence. This is the module imported by the CLI script.

You also need to create the following at the project root level:

- **scripts/generate_visualizations.py** – is the CLI entry-point. Run it from the project root with `python scripts/generate_visualizations.py` to regenerate all charts. It accepts an optional `--data` flag to point at a different CSV file.
- **notebook/lab12_visualization.ipynb** – is the main submission notebook demonstrating all lab steps interactively: setup, each static chart, each interactive chart, the automated generator, and the Assignment 12 solution.

Inside **outputs/**, two new folders store all generated chart files. **outputs/visualizations/static/** holds the PNG and PDF exports produced by matplotlib and seaborn so you do not need to commit these files to version control as they are fully reproducible by running the script. **outputs/visualizations/interactive/** holds the HTML files produced by Plotly and open any of them directly in a browser to interact with the chart.

Part 1 - Install Required Libraries and Tools

The visualization stack relies on three complementary libraries, each suited to a different use case:

Library	Version	Primary Role
matplotlib	≥ 3.8	Low-level 2-D plotting; object-oriented API; PNG/PDF export
seaborn	≥ 0.12	Statistical chart templates built on top of matplotlib
plotly	≥ 6.0	Interactive HTML charts; Plotly Express high-level API and Graph Objects
kaleido	≥ 0.2	Static image export for Plotly figures (PNG/SVG/PDF)
pandas	≥ 2.1	Data loading, groupby aggregations, DataFrame manipulation
numpy	≥ 1.26	Numerical operations and array utilities

To install the visualization-specific packages run:

```
pip install matplotlib seaborn plotly kaleido pandas numpy
```

To install specific minimum versions run:

```
pip install "matplotlib>=3.8" "seaborn>=0.12" "plotly>=6.0"
"kaleido>=0.2" "pandas>=2.1" "numpy>=1.26"
```

Part 2 - Understanding Data Visualization

A visualization communicates a data insight faster than a table of numbers ever could. Before writing any code, the most important decision is which chart type matches the question you are asking.

2.1 matplotlib – Object-Oriented API

matplotlib is Python's foundational plotting library. This lab uses the object-oriented API exclusively. Every chart starts with an explicit Figure and Axes object so every visual element can be configured precisely. The Agg backend must be set before importing pyplot. It allows `savefig()` to work in scripts and notebooks without opening a display window.

2.2 seaborn – Statistical Charts

seaborn is built on top of matplotlib and provides ready-made statistical chart functions (histplot, boxplot, scatterplot, heatmap) with much less boilerplate.

Note: since seaborn 0.12, passing a palette without a `hue=` parameter is deprecated. For boxplots and similar charts, always pass `hue='column_name'` with `legend=False` when you want colour by category.

2.3 Plotly – Interactive Charts

Plotly produces interactive web charts saved as self-contained HTML files.

It has two APIs:

1. `plotly.express (px)` for one-line charts with hover, zoom, and legend toggle and
2. `plotly.graph_objects (go)` for lower-level control and multi-panel layouts with `make_subplots()`.

Every chart is saved with `fig.write_html()` and can be opened in any browser without a server:

[2.4 Setting Up the Package](#)

In `src/visualization/` you need to add the `__init__.py` file first. This file makes the visualization folder a Python package so its modules can be imported in other parts of the project. After that, open `notebook/lab12_visualization.ipynb` and run the first setup cell. It adds `src/` to the Python

path, sets the Agg backend, imports all libraries, creates the output directories, and loads the cleaned dataset from previous weeks.

Part 3 – Static Bar and Line Charts with Matplotlib

In this part you need to create the file **src/visualization/static_charts.py** and add the first two chart functions to it. This module will hold eight static chart functions and write them to PNG and PDF files.

3.1 Top 10 Movies by Revenue – Horizontal Bar Chart

Add the first function to **static_charts.py**. A horizontal bar chart is used here because the category labels (movie titles) are long strings that read more naturally from left to right. Revenue values are converted to USD billions and inline annotations are added so the reader does not have to look up each bar on the x-axis. In the notebook, call the function directly and run `plt.show()` to preview the figure inline before saving.

3.2 Average Rating Over Years – Dual-Axis Line Chart

Add a second function to **static_charts.py**. This chart overlays two related but differently scaled series on the same x-axis: the number of movies released per year (bars, left y-axis) and the mean audience rating per year (line, right y-axis). Use `ax.twinx()` to create the second y-axis that shares the x-axis. In the notebook, demonstrate this chart after the bar chart. In my output, the chart shows that the number of movies in the dataset has increased towards recent years and that the average rating stays roughly between 6 and 8 throughout the period.

Part 4 – Statistical Charts with Seaborn

Continue adding functions to **src/visualization/static_charts.py**. This part uses seaborn's statistical chart functions. All functions follow the same pattern: create a `fig, ax` with `plt.subplots()`, pass `ax=` to the seaborn function, configure labels and title, then call `_save()`.

Part 5 – Multi-Panel Dashboard Layout

Add the final function to **src/visualization/static_charts.py**. This function creates a 2×2 subplot figure that combines the four most informative views on a single canvas. Use `plt.subplots(2, 2)` to create the grid and address each panel by its row-column index. This is a common pattern for executive dashboards where a single shareable image must tell the whole story.

In the notebook, call **plot_dashboard_subplots(df)** and display the result inline. You should see four panels on one canvas. After this, **static_charts.py** is complete with 8 functions. You can run

a quick test by importing and calling each function individually in the notebook to confirm all 8 PNG and PDF files are created under **outputs/visualizations/static/**.

[Part 6 – Interactive Charts with Plotly Express](#)

Now create a new file **src/visualization/interactive_charts.py**. This module holds all five interactive chart functions. In the notebook, call each of these four functions and use `fig.show()` to display the chart inline. Hover over individual points to see the tooltip, click genre names in the legend to hide or show them, and use the box-select tool to zoom in. Open the saved HTML files in a browser to confirm they work as standalone documents.

[Part 7 – Interactive Multi-Layout Dashboard](#)

Add the final function to **src/visualization/interactive_charts.py**. This function builds a 2×2 interactive dashboard using `plotly.graph_objects` and `make_subplots()`. Unlike Plotly Express, Graph Objects are used here because `make_subplots()` requires you to add each trace manually with `fig.add_trace(row=, col=)`. In the notebook, call `interactive_multi_layout(df)` and display the result. You should see four connected panels. Each panel is individually zoomable and hoverable. After this, `interactive_charts.py` is complete with 5 functions.

Part 8 – Automated Chart Generation

Now create two more files that wire everything together into a single runnable command.

[8.1 Creating chart_generator.py](#)

Create **src/visualization/chart_generator.py**. This module is the orchestrator: it loads the cleaned dataset, iterates over all static and interactive chart functions, calls each one, and prints progress. It imports from the two modules you just created.

[8.2 Creating the CLI Script](#)

Create **scripts/generate_visualizations.py**. After running the script you should find 16 files (8 PNG + 8 PDF) in **outputs/visualizations/static/** and 5 HTML files in **outputs/visualizations/interactive/**. Open any HTML file directly in a browser to interact with the chart.

[Part 9 – Integrate into run_pipeline.py](#)

Update the existing **src/pipeline/run_pipeline.py** to include a visualization step.

You can then call `run_visualizations_pipeline()` at the bottom of `run_pipeline.py` after `run_analytics_pipeline()` to make the visualization step part of the full pipeline run.

Part 10 – Document Visualization Choices

Add a final section to your notebook (and include it in your submission document) that explains why each chart type was chosen for each analysis question. This is a required part of the assignment. For each chart, explain: what question it answers, why that chart type is the correct choice for that data, and what encoding decisions were made (colour, size, axis). The guiding principle is to match the chart type to the statistical nature of the data: use bars for comparing discrete categories, lines for trends over time, scatter plots for relationships between two continuous variables, histograms for distributions, box plots for comparing distributions across groups, and heatmaps for pairwise correlation matrices.

Assignment Task:

1. Static Charts (35%)
 - Create **src/visualization/static_charts.py** with 8 matplotlib and seaborn chart functions
 - Each function must accept a DataFrame and an output directory, draw a figure using the object-oriented API, and save it as both PNG (300 dpi) and PDF
 - Implement at least 8 charts
 - Apply seaborn themes and palettes consistently across all static charts
2. Interactive Charts (25%)
 - Create **src/visualization/interactive_charts.py** with 5 Plotly chart functions
 - Each function must save a self-contained interactive HTML file
 - Implement at least 5 charts
 - Include rich hover tooltips with at least 3 data fields in each chart
3. Module Architecture (15%)
 - Create **src/visualization/__init__.py** that re-exports all chart functions
 - Create **src/visualization/chart_generator.py** as the orchestrator module
 - Create **scripts/generate_visualizations.py** as the CLI entry-point with an optional `--data` flag
 - Update **run_pipeline.py** with a new **run_visualizations_pipeline()** step
4. Notebook Demonstration (15%)
 - Demonstrate all charts in **notebook/lab12_visualization.ipynb**

- Include written Markdown explanations after each chart describing what it shows and why that chart type was chosen
 - Run all cells from top to bottom without errors
5. Visualization Choices Documentation (10%)
- Document the visualization choice for every chart type implemented
 - Explain why each chart type was selected for the underlying data characteristic
 - Reference Tufte's principles of graphical excellence in the justification